EpiScope: A Big Data Surveillance, Forecasting and Response-Planning Platform for Seasonal, Tropical and Epidemic Diseases in Bangladesh

MD Naim Uddin Mozumdar
ID 0222220005101190
Data Engineer, team lead

Shamraj Deb Nath
ID 0222220005101180
Analytics Engineer

Syed Nasibur Rahaman Ratul
ID 0222220005101189
Visualisation Engineer

Puja Dey
ID 0222220005101191
Testing and QA Engineer

CSE 4345 Big Data Analytics, Complex Engineering Problem · Group 2 · Premier University, Chittagong · Spring 2026
Repository: `https://github.com/naimuddin1190/episcope` (tag v1.0) · Video: `https://youtu.be/cjTeE0yruFo`

Abstract

Bangladesh faces strongly seasonal dengue, diarrhoeal disease, influenza-like illness (ILI) and malaria, but the data needed to respond early is spread across facility reports, laboratories, weather records and stock ledgers. EpiScope is a medallion lakehouse built with Apache Spark and Parquet that ingests heterogeneous batch and incremental feeds, enforces 20 automated quality rules, de-identifies patients and builds epi-week marts. On top of these it detects outbreaks per district, forecasts cases 1 to 6 weeks ahead with an interpretable Poisson model, converts forecasts into supply demand, allocates a fixed budget with a linear programme that trades efficiency against equity, and derives a seasonal preparedness calendar. A D3.js dashboard with five linked views serves only aggregated, small-cell-suppressed data (2.2 MB of JSON). On 28.6 million synthetic encounters with injected ground truth, the chosen detector found 87 % of 101 outbreak episodes at 0.99 false alerts per district-year; one-week forecasts cut MAE by 57 % (dengue) and 31 % (diarrhoea) against a seasonal-naive baseline; and the optimiser left 3.9 % of need unmet against 24.1 % for population-proportional allocation at the same budget. The full pipeline runs at SF 1 in 41 minutes on an 8 GB machine, after column pruning cut the gold stage by 67 %.

I. Introduction and problem analysis

EpiScope turns scattered surveillance, weather and stock data into weekly district alerts, forecasts and a supply plan that officials can defend.

Burden and seasonality. Dengue peaks after the monsoon, with the 2023 epidemic causing 321,179 hospitalisations and 1,705 deaths, the worst year on record [11]. Diarrhoeal disease rises before the monsoon and after extreme rain, ILI rises in the rainy season, and malaria is concentrated in the Chittagong Hill Tracts.

The data problem. Facility reports, laboratory results, weather and stock ledgers differ in format, timeliness, quality and granularity. Reports arrive 0 to 21 days late, facilities fall silent and back-fill, records are duplicated and codes are invalid. Today these are analysed after the fact, so supplies and vector-control campaigns start late.

Client objectives. The Ministry asked for (i) early detection of unusual rises per district and disease, (ii) short-term forecasts with uncertainty, (iii) a defensible allocation of a fixed budget of supplies, and (iv) a seasonal preparedness calendar. Non-functional needs are trust and explainability, robustness to late and incomplete data, privacy, modest infrastructure (one laptop) and scalability.

Contributions.

- A seeded synthetic generator for encounters, facilities and stock, driven by real Open-Meteo weather, with injected outbreaks, a back-fill artefact and realistic data defects.
- An idempotent bronze, silver and gold lakehouse in Spark and Parquet with 20 quality rules, quarantine with reason codes, lineage and salted hashing.
- Two outbreak detectors, a per-district forecasting model, an LP allocator and an STL calendar, each evaluated against a baseline.
- A five-view linked D3.js dashboard with a what-if panel and small-cell suppression.
- A 52-test automated suite, a scalability study at SF 0.25, 0.5 and 1, and two Spark optimisations with Spark UI evidence.

II. CEP attribute analysis and conflict resolution

Every CEP attribute shaped a concrete design decision, and each of the five conflicts was settled with an experiment or an explicit policy.

C1 Sensitivity vs specificity. We swept thresholds 1 to 6 for both detectors and plotted sensitivity against false alerts per district-year (Table 6.1, from `detection_tradeoff_curve.csv`). The operating point is the highest sensitivity with at most one false alert per district-year. EARS C2 never reached that limit (4.6 false alerts per district-year even at h = 6), because it reacts to every seasonal rise. Historical limits at h = 6 did: 87 % sensitivity (88 of 101 episodes) at 0.99 false alerts per district-year, median 19 days to detection. Officers can move the threshold with a dashboard slider.

C2 Granularity vs privacy. We publish at district level only. Any count from 1 to 4 is hidden, and complementary suppression hides a second cell so that a hidden value cannot

Attribute	How it appeared	Our decision
P1 Depth of knowledge	Distributed processing, columnar storage, surveillance statistics, forecasting, optimisation, visual encoding, privacy	Spark and Parquet medallion lake; EARS C2 and historical limits; Poisson GLM; LP; D3.js; suppression
P2 Conflicting requirements	C1 to C5 below	Each resolved with numbers or a stated policy
P3 Depth of analysis	No single right threshold, model or objective	Threshold trade-off curve, rolling-origin backtest, 25-scenario budget and equity grid
P4 Unfamiliar issues	Reporting delays, back-fill artefact, duplicates, orphan facility keys, Dhaka skew	Onset-week counting and nowcasting, checksum idempotency, rules R16 and R04, AQE
P5 Beyond standard codes	No standard for a national allocation plan	IDSR thinking, ICD-10 coding and data-protection principles combined with our own LP
P6 Stakeholders	Ministry, epidemiologists, district officers, hospitals, vector control, public, regulator	Role-specific views: map and table for officers, banded forecasts for hospitals, suppression for the regulator
P7 Interdependence	Quality feeds detection, which feeds forecasts, the optimiser and the dashboard	Lineage on every gold row, DAG with retries, upper forecast quantile carried into demand

be recovered from division or national totals. Upazila maps would hide most cells for the rarer diseases, so the district is the most useful safe level.

C3 Efficiency vs equity. Fig. 2 shows demand met against coverage of the worst-served district for 5 budgets and 5 equity floors α. At the default budget (60 % of full need) moving α from 0 to 0.5 raised the worst-served district's coverage from 75.5 % to 77.8 % while weighted unmet demand rose only from 0.3570 to 0.3574 (+0.1 %), so we recommend α = 0.5. Without a floor at a tight budget (20 %), 51 of 64 districts received nothing. In policy terms, α guarantees every district a minimum share of each resource in proportion to its population.

C4 Freshness vs cost. Supply orders and campaigns are decided weekly, and reports arrive 0 to 21 days late. A daily incremental batch that rebuilds only the affected epi-years is fresh enough; streaming would add cost and complexity without changing any decision.

C5 Accuracy vs interpretability. A per-district Poisson GLM with named covariates (recent weeks, same week last year, season, rainfall, temperature) beats the seasonal-naive baseline by 57 % (dengue) and 31 % (diarrhoea) in one-week MAE. Its drivers can be explained to officials; a black-box model would be harder to trust for a small gain.

III. Architecture and design decisions

EpiScope is a single-machine medallion lakehouse: files land unchanged in bronze, are cleaned and de-identified in silver, aggregated into epi-week marts in gold, and only aggregated, suppressed JSON reaches the dashboard.

Storage and format. Every layer is Parquet with Snappy compression. Bronze is partitioned by `batch_id`, silver by `epi_year` and `batch_id`, gold by `epi_year`. All tables are registered in the Spark SQL catalog (`episcope_bronze.*`, `episcope_silver.*`, `episcope_gold.*`).

Schema recognition. Files are recognised by their header or content, never by name, so the hidden 14-day drop can arrive with any file names.

Idempotency. A SHA-256 manifest skips any file already loaded, silver uses dynamic partition overwrite, and gold de-duplicates by `encounter_id` across batches. Re-delivering a loaded file under a new name produced "no new files", "no new batches" and "nothing to rebuild" (Section 6.6).

Option	Why not (now)
Delta Lake or Iceberg	Time travel would help backtests; deferred to keep setup to pip installs
HDFS or MinIO	Not needed on one laptop; Parquet on local disk is enough
Airflow or Prefect	Heavy for a nine-stage linear DAG; Makefile plus Python retries suffice
Kafka and Structured Streaming	No weekly decision needs sub-day freshness (C4)
Prophet or LSTM	Less interpretable for a small expected gain (C5)

Late data. Silver records which epi-years a batch touched, and gold rebuilds only those years, so a record up to 21 days late updates exactly its own weeks.

Orchestration. A Python DAG runs nine stages with retries (`bronze_ingest` to `serving_export`). One command, `make all`, rebuilds everything from a clean clone; `make update` loads a new drop.

Alternatives considered.

IV. Data generation, quality, lineage and privacy

At SF 0.5 the generator produced 14,306,952 encounter rows and 1,747,482 stock rows; silver quarantined 486,386 encounters (3.4 %) and 5,270 stock rows (0.3 %), each with a reason code.

4.1 Generator. Real public data covers 64 districts (geoBoundaries), census population by age and sex, and daily Open-Meteo weather (S1 to S3). The seeded generator simulates 957 facilities, encounters and daily stock for 2021 to 2025 (S4 to S6), scaled by SF (SF 1 is about 28.5 million encounters). Seasonality follows von Mises curves, dengue responds to rainfall with a 4 to 8 week lag, diarrhoea rises after extreme rain, and noise is negative-binomial. Injected ground truth includes local outbreaks, a slow-growing diarrhoeal outbreak in Cox's Bazar (119 days), a nationwide respiratory surge (70 days) and a 2023-style dengue epidemic in 14 districts. One Rajshahi facility goes silent for 21 days and then back-fills, the artefact a naive detector should flag. Defects include about 2 % duplicates, 0.5 % orphan facility keys and

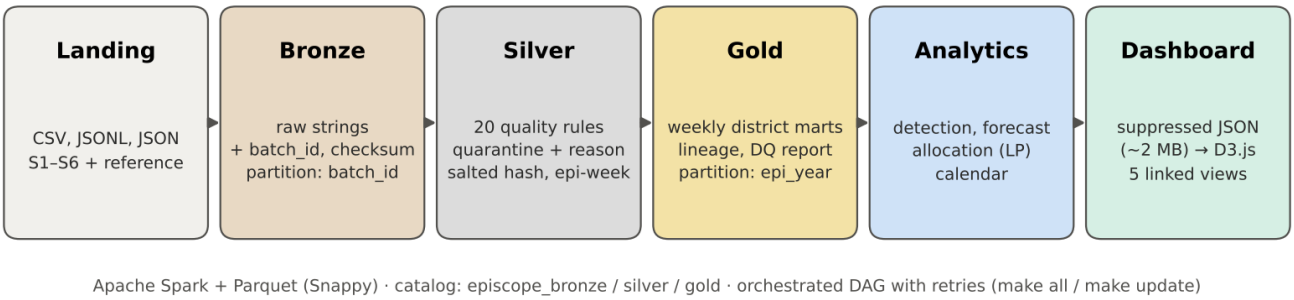


Figure 1. EpiScope medallion architecture: files land in bronze unchanged, are cleaned and de-identified in silver, aggregated into weekly marts in gold, analysed, and served as suppressed JSON to the D3.js dashboard.

Reason code	Meaning	Rows
R16_DUPLICATE_IN_BATCH	Same encounter delivered twice	27,673
R04_FACILITY_UNKNOWN	Facility not in registry	7,116
R02_PATIENT_ID_NULL	Patient ID missing	2,410
R05_ONSET_INVALID	Onset date missing or malformed	2,407
R12_SEX_INVALID	Sex not M or F	2,383
R11_AGE_INVALID	Age outside 0 to 110	2,369
R13_ICD10_INVALID	Invalid ICD-10 code	2,328
R07_ONSET_AFTER_REPORT	Onset after report date	2,110

1 % invalid values; reporting delays run 0 to 21 days, with fewer reports on weekends and holidays.

4.2 Quality rules. Silver applies 16 encounter rules (R01 to R16) and 4 stock rules (S01 to S04): missing keys, referential integrity with the facility registry, date validity and order, implausible delays, future or too-old dates, age, sex, ICD-10 and flag validity, outcome codes and in-batch duplicates. A failing row goes to `silver/quarantine` with its first reason, all reasons and the raw record, and a per-batch report (rows in, rows valid, pass rate per rule) is written to `gold/dq_report` and shown in the dashboard.

Quarantine reasons in the SF 0.05 development run (total 48,796 rows):

4.3 Lineage. Every bronze row carries `batch_id`, `_source`, `_load_ts`, `_file_name` and `_file_checksum`; every gold row carries the sorted list of `bronze_batch_ids` it came from; and `gold/batches` links each batch to files, checksums and row counts.

4.4 Privacy. `patient_id` is replaced in silver by a salted SHA-256 hash; the salt lives only in `.env`, which is never committed. Age becomes six bands and raw identifiers never leave bronze. The serving layer publishes district counts only, with primary suppression of 1 to 4 and complementary suppression (Section 2, C2). Automated tests check that no 1 to 4 count is published or recoverable.

V. Analytics methods

All five methods run per district and disease in Spark, either with window functions or with grouped `applyInPandas`, so they scale with the number of series rather than with one machine's memory.

5.1 Outbreak detection. Two established methods are computed for every district, disease and week.

- *EARS C2* [1] with Spark window functions: the baseline is the mean and standard deviation of weeks t−9 to t−3 (seven weeks, with a two-week guard band so an ongoing outbreak does not raise its own baseline). The statistic is $(x_t - \text{mean}) / \max(\text{sd}, \sqrt{\text{mean}}, 1)$, and it needs at least five baseline weeks.
- *Historical limits* (Farrington-style) [2] with `applyInPandas`: the baseline is the same week ±2 in each of up to three previous years, scaled to the recent level (clipped to 0.5–1.3).

Two P4 issues are handled explicitly. Cases are counted by **onset week**, so a facility back-filling three weeks of reports on one day does not create a spike. Recent weeks that are not yet mature are **nowcast**: counts are divided by the share of cases historically reported by that delay, taken from `gold.delay_distribution`. An alert needs the statistic above the threshold and at least 5 cases.

5.2 Forecasting. A direct multi-horizon Poisson GLM is fitted per district, disease (dengue, diarrhoea) and horizon h = 1 to 6 weeks. All features are known at the forecast origin: log cases this week and last week, log 4-week mean, log cases in the target week one year earlier, sine and cosine of the target week, rainfall summed over weeks t−3..t and t−7..t−4, and mean 4-week temperature. Prediction intervals come from a negative-binomial distribution whose dispersion is estimated from training residuals, giving 7 quantiles from 2.5 % to 97.5 %.

5.3 Demand conversion. Forecast cases over the next 4 weeks at the 90 % quantile are converted to resources. Admission rates are estimated from the last 52 mature weeks per district and disease; per-patient consumption is a stated planning assumption.

5.4 Optimisation. We solve the course LP with PuLP and CBC. Decision variables are allocations $x_{d,r} \geq 0$ and unmet demand $u_{d,r} \geq 0$ for district d and resource r. The objective minimises weighted unmet demand $\Sigma\, w_d\, u_{d,r}$, normalised per resource, where $w_d = 1 + 2\cdot$(active alert) $+ 1\cdot$(bed occupancy above 85 %). Constraints are unmet demand $\geq$ forecast demand − stock − allocation, total cost $\leq$ budget, the equity

Assumption	Value
IV saline per admission	4 bags (dengue), 6 bags (diarrhoea)
NS1 kits per dengue case	1.3
ORS per diarrhoea case	4 sachets
Unit cost (BDT)	IV saline 90, NS1 kit 350, ORS 7
Default budget	60 % of the cost of all unmet need

floor $x_{d,r} \geq \alpha_{\cdot d} \cdot \sum x_r$ (p_d = population share), and division transport capacity at 110 % of the division's volume share. The baseline allocates the same budget in proportion to population.

5.5 Preparedness calendar. For each division and disease, an STL decomposition of the weekly series gives the seasonal component. The season start is the week the seasonal curve first rises above its mean; supplies should be pre-positioned 4 weeks earlier and prevention campaigns started 6 weeks earlier. `For dengue the season starts in mid-June (8 June in five divisions, 15 June in Barishal, Chattogram and Sylhet), so supplies are due by 11–18 May and campaigns by 27 April–4 May; for diarrhoea the season starts on 16 February nationwide.`

VI. Evaluation

Every method was evaluated against a baseline on held-out or ground-truth data; the engineering results come from runs on one 8 GB, 12-thread Windows PC under WSL2 (10 CPUs and 6 GB given to Linux, Spark driver 4 GB).

6.1 Detection (AN-2). Evaluated on mature weeks from the second year on (101 injected episodes), against ground truth the detectors never read. At the chosen point (historical limits, h = 6, at least 5 cases) sensitivity was 87 % at 0.99 false alerts per district-year, median 19 days from outbreak start to the end of the first alerting week. EARS C2 needed far more false alerts for similar sensitivity (90 % at 21.8 per district-year at h = 3).

The weak spots are honest limitations: small local outbreaks and malaria (low counts) are often below the threshold, the broad 2023 dengue epidemic rises like a strong season and is caught late, and the slow-growing outbreak is found only after 91 days.

6.2 Reporting artefact (AN-3). For the Rajshahi facility (F24003) that went silent for three weeks and back-filled on 25 February 2024, a naive detector on report weeks reached 4.02 SD in the back-fill week, a false alarm at the usual threshold of 3. EpiScope, counting by onset week, reached only 0.67 SD: no alarm.

6.3 Forecast backtest (AN-5). Rolling origin over the last 52 weeks (3,328 forecasts per disease and horizon), refitted every 4 weeks, against seasonal-naive (same week last year). Skill = 1 − model / baseline.

The model beats the baseline at every horizon, skill falls as the horizon grows, and its 80 % intervals cover 80–86 % of outcomes, so the bands are honest rather than too narrow.

6.4 Allocation (AN-7, AN-8). Planning covers the 4 weeks after the 31 December 2025 data cutoff; meeting all need would cost BDT 2,931,057. At the default budget (60 %, BDT 1,758,634) and α = 0.5:

Across the 25-scenario grid (budget 20–100 %, α 0–0.5) the optimiser beats population-proportional allocation at every budget below 100 %. (Fig.~2)

6.5 Scalability (ENG-2). Every run used the same settings: WSL2 with 10 CPUs and 6 GB RAM on an 8 GB, 12-thread PC, Spark driver 4 GB, optimisations on. The requirement is SF 1 in at most 45 minutes on a 16 GB laptop.

The pipeline meets the 45-minute target on half the RAM the requirement assumes. Bronze and gold grow roughly linearly; analytics stays flat because it works on gold aggregates. Silver grows faster than the data (2.1× from SF 0.5 to 1) because its de-duplication shuffle no longer fits in 6 GB: during the SF 1 run Spark used 571 % CPU with only 95 MB of RAM free and 634 MB of swap in use, so memory, not CPU, is the limit.

6.6 Spark optimisations (ENG-3). Three optimisations, each with before and after evidence:

1. **Column pruning before de-duplication (gold).** Gold de-duplicated all 27 silver columns by `encounter_id` before aggregating, shuffling and caching hashes, checksums and file names it never used. Selecting the 12 needed columns first left every gold total unchanged (67,840 rows; 682,427 cases; 138 deaths; identical before and after) and all 52 tests passed. At SF 0.5, gold fell from 771 s to 260 s. Raising CPUs from 4 to 10 had left SF 1 gold unchanged (1,554 s vs 1,544 s), so the gain comes from the code change, not the CPU change.
2. **Broadcast joins (gold).** With optimisation off, the weekly mart joined the 64-row district tables with SortMergeJoin, which sorts both sides first. With it on, Spark broadcast the small side (64 rows, 8 MiB, 17 ms) and used BroadcastHashJoin with no sort stages (Spark UI, Figs. 3–4).
3. **Adaptive Query Execution** with skew-join handling and partition coalescing, switched on together with (2).

Silver did not change because its facility join is already an explicit broadcast in the code. The Stages view shows the remaining bottleneck: the de-duplication window in silver shuffles about 1.15 GB and takes 4.3–4.8 minutes at SF 0.5 (Figs. 5–6).

6.7 Automated tests (ENG-4). The suite has 52 pytest tests, all passing: quality rules, de-identification, schema contracts, suppression, optimiser constraints, checksum idempotency and schema recognition, forecast intervals and look-ahead safety, EARS C2 behaviour, and detection scoring.

VII. Dashboard design rationale and usability findings

The dashboard answers one question per view for a district officer, links every view to one shared selection, and loads about 2 MB of aggregated, suppressed JSON, never line-level data.

Linking. Clicking a district on the map or table updates every view; brushing the time series sets the period for the map and heatmap. Extra panels show the preparedness calendar, data quality per batch and model evaluation.

What-if panel. Three sliders change the alert threshold, the budget (20–100 % of need) and the equity floor α; allocations

Breakdown	Episodes	Detected	Sensitivity	Median days to detect
Dengue	20	14	70 %	43.5
Diarrhoea	11	8	73 %	18.5
ILI	64	64	100 %	18
Malaria	6	2	33 %	15
Nationwide respiratory surge	64	64	100 %	18
2023-style dengue epidemic	14	11	79 %	49
Local outbreaks	22	12	55 %	17.5
Slow-growing (Cox's Bazar)	1	1	100 %	91

Disease	h	MAE model	MAE naive	MAE skill	WIS skill	80 % coverage model / naive
Dengue	1	10.1	23.7	57 %	67 %	85 % / 72 %
Dengue	2	12.0	23.7	49 %	64 %	86 % / 72 %
Dengue	4	12.4	23.7	48 %	62 %	83 % / 71 %
Diarrhoea	1	86.2	125.1	31 %	40 %	83 % / 79 %
Diarrhoea	2	94.6	125.1	24 %	34 %	82 % / 79 %
Diarrhoea	4	103.8	125.1	17 %	28 %	80 % / 79 %

Method	Unmet need	Weighted unmet	Worst district covered	Coverage Gini
Optimiser (LP)	3.9 %	0.357	77.8 %	0.032
Population-proportional	24.1 %	0.453	70.9 %	0.055

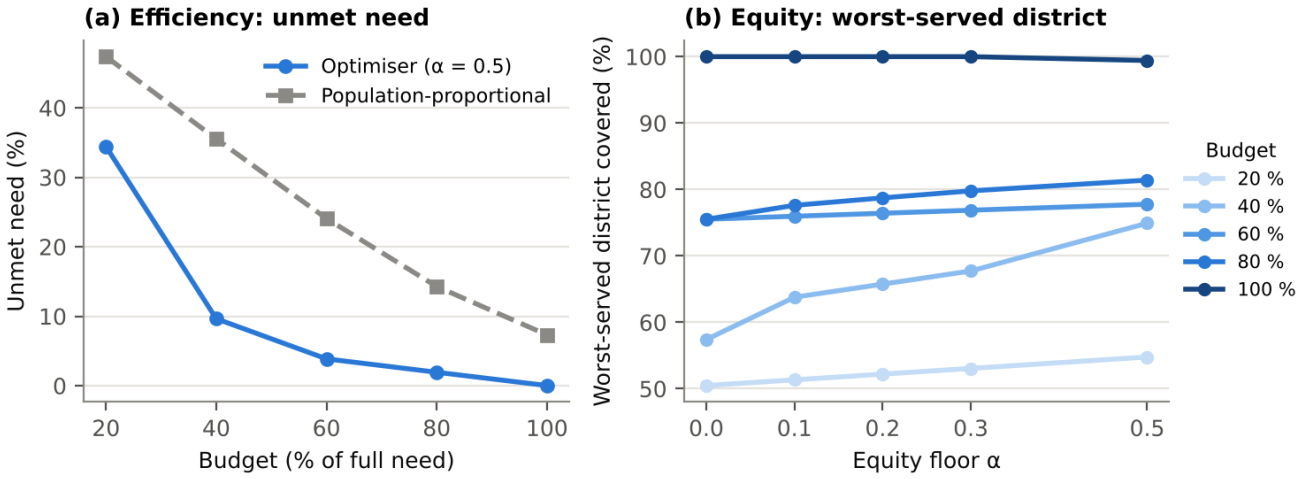


Figure 2. Efficiency–equity trade-off (C3): (a) unmet need by budget, optimiser ($\alpha = 0.5$) vs population-proportional; (b) coverage of the worst-served district by equity floor α for five budgets.

Stage (s)	SF 0.25	SF 0.5	SF 1
generator (synthetic data, not the pipeline)	75	140	325
bronze_ingest	146	263	397
silver_transform	398	686	1,438
gold_build	156	305	509
analytics (detection to serving)	117	132	131
Pipeline total	**817 (13.6 min)**	**1,386 (23.1 min)**	**2,475 (41.3 min)**
Including generator	893	1,526	2,801 (46.7 min)
Encounter rows	about 7.1 M	14.3 M	28.6 M
Silver size (MB)	593	1,159	2,289
Dashboard JSON (MB)	2.1	2.1	2.2

come from a precomputed 25-scenario grid so the page stays instant.

Honesty and accessibility (VIS-6). A colour-blind-safe palette, uncertainty bands on every forecast, hatched suppressed cells with a legend, a "data up to" freshness stamp on every view, and a permanent banner stating that the data is synthetic.

Usability test. On 1 November 2026 two participants

Stage, SF 0.5	Optimisations off	On	Change
bronze_ingest	223.2 s	208.5 s	−7 %
silver_transform	553.4 s	565.3 s	+2 % (noise)
gold_build	259.9 s	221.6 s	−15 %
Total	1,036.5 s	995.4 s	−4 %

View	Question it answers	Encoding
1 Choropleth map	Where is incidence high, and where are alerts?	Sequential colour per 100,000; triangle for an alert in the last 4 weeks; hatching for suppressed cells
2 Time series	How is this district changing, and what comes next?	Weekly cases, alert threshold line, forecast median with 50 % and 80 % bands, brush to pick a period
3 Heatmap	Which weeks and districts stand out?	District × week grid, same colour scale
4 Allocation	Who gets which supplies, and why?	Bars of optimiser vs population-proportional allocation, unmet need
5 Ranked table	Which districts need attention first?	Sortable table of incidence, alerts, forecast and stock

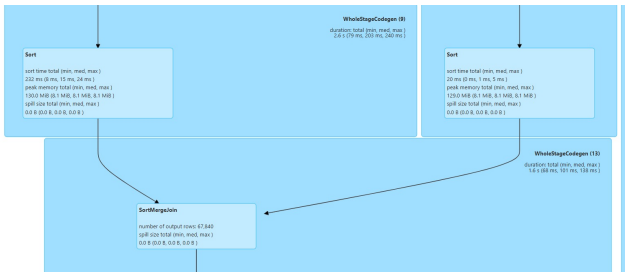


Figure 3. Optimisations off: the weekly mart joins the district table with SortMergeJoin after sorting both sides.

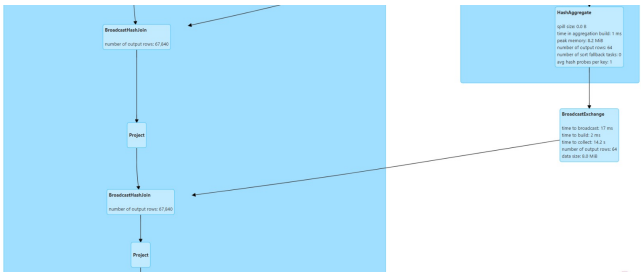


Figure 4. Optimisations on: the 64-row district table is broadcast (8 MiB, 17 ms) and joined with BroadcastHashJoin; no sort stages.

with no computer-science background did five tasks on the dashboard without help, thinking aloud, and were timed with a stopwatch (give-up limit 2 minutes). P1: bank officer, about 35, uses a computer daily. P2: business owner, about 45, uses a computer sometimes. No names were recorded.

Both called the dashboard "fairly easy" overall.

Findings. The map, the alert triangles, the privacy hatching and the what-if slider worked for both users (8 of 8 attempts). The time series was the weak view: only the daily computer user understood the forecast band, and neither stated the recent trend clearly, so uncertainty is shown honestly but not yet explained well enough for every non-specialist (VIS-6). The less frequent computer user struggled most, which matters because district officers vary in data literacy.

Change. We added a plain-language caption under the time series: "The line is cases per week. The shaded band on the right is where the next weeks will probably fall (a likely range, not an exact number). The grey strip below only chooses the period." This targets both observed confusions: the band and the period selector (commit "VIS-6: plain-language caption for the forecast band after usability test"). A "rising / falling compared with 6 months ago" label for the selected district is left as future work (Section 9).

VIII. Ethical, Societal and Sustainability Considerations

The main risks are re-identification, stigma, harm from wrong alerts and unfair allocation; each has a design control, and the remaining risk is stated openly.

Purpose limitation and data minimisation. Only the fields needed for surveillance are processed; the dashboard carries no line-level data and the salt never enters the repository.

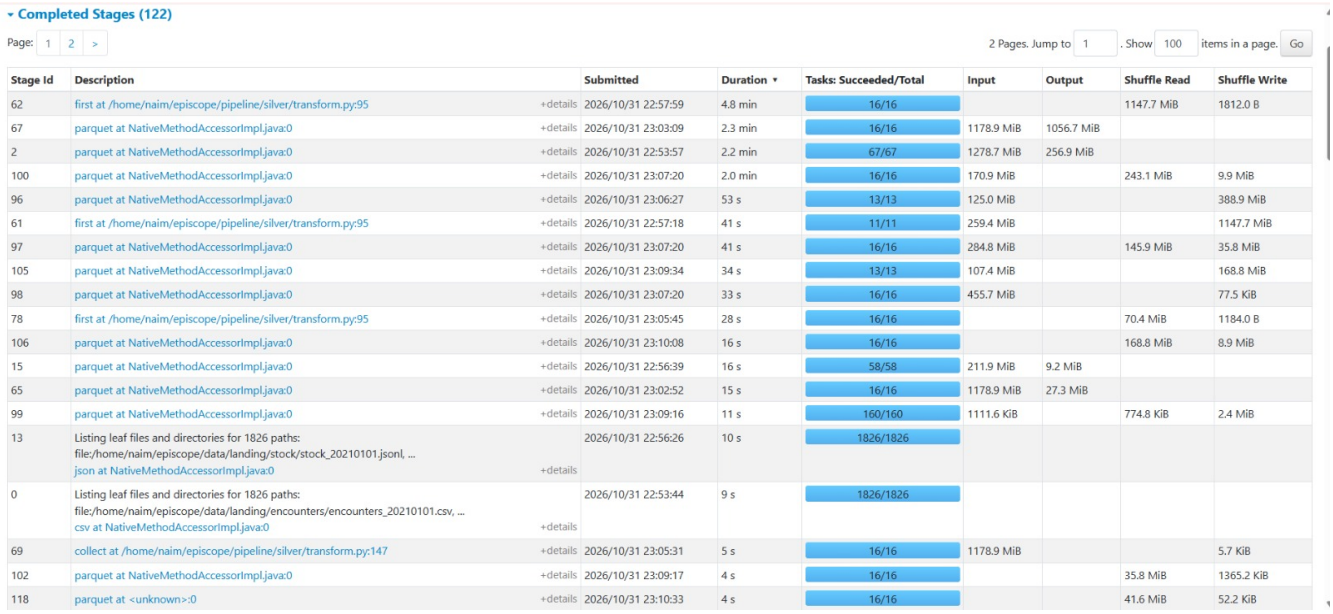
Sustainability. The whole system runs on one laptop with open-source software and no paid cloud, so a district office can run it. A full SF 1 rebuild takes about 41 minutes on an 8 GB PC; the daily incremental load touches only new files and affected years, which keeps energy use small.

Synthetic data. All patient-level data is synthetic. Results show that the methods work under realistic conditions, not that these exact numbers hold for Bangladesh.

IX. Limitations and Future Work

The largest limitations are synthetic epidemiology, a memory-bound silver stage on an 8 GB machine, and a single-period allocation model.

- **Synthetic data.** Seasonality, outbreaks and defects are modelled, not observed; real DHIS2 data would test the methods properly.
- **Performance.** Silver's de-duplication shuffle dominates at SF 1 on 8 GB RAM. Next steps: de-duplicate on a narrow key projection, salt the Dhaka key, and test on a 16 GB machine or Databricks Free Edition.
- **Granularity.** District, not upazila, level (C2); population structure is approximate.
- **Optimiser.** One period, no transport lead times, no multi-week stock carry-over.
- **Detection.** Slow-growing outbreaks are found late, and a broad epidemic can be partly absorbed by a level-scaled baseline. Small local outbreaks (55 %) and malaria (33

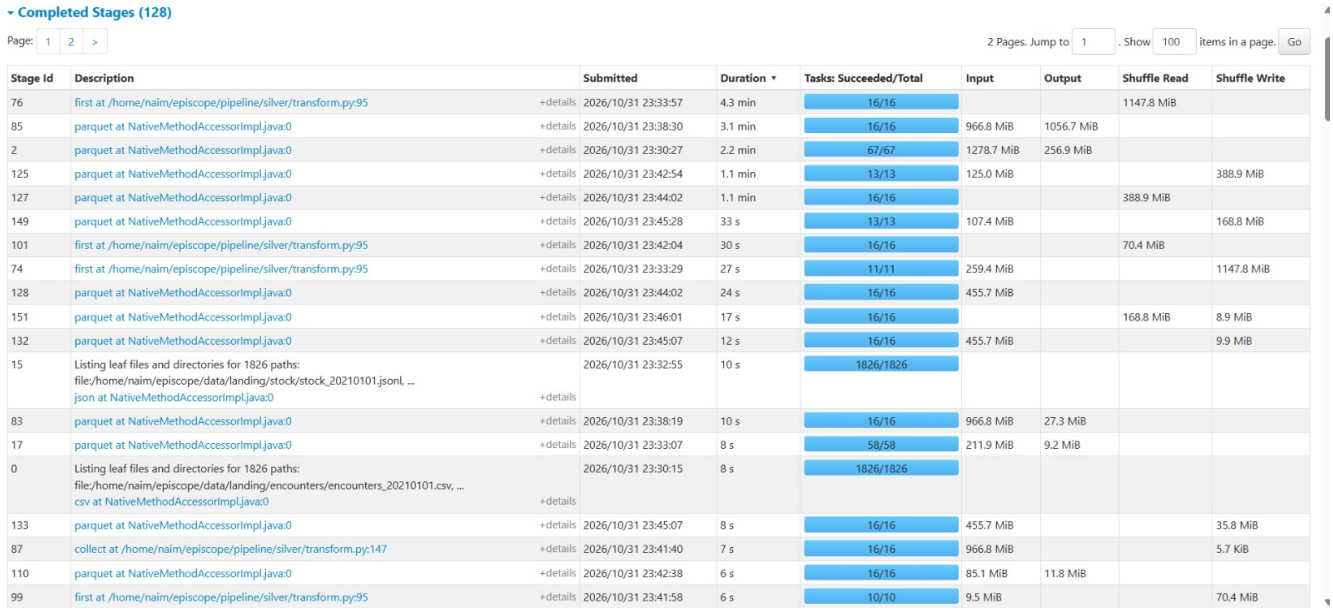


▾ Completed Stages (122)

Page: 1 2 > 2 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
62	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 22:57:59	4.8 min	16/16			1147.7 MiB	1812.0 B
67	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:03:09	2.3 min	16/16	1178.9 MiB	1056.7 MiB		
2	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 22:53:57	2.2 min	67/67	1278.7 MiB	256.9 MiB		
100	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:07:20	2.0 min	16/16	170.9 MiB		243.1 MiB	9.9 MiB
96	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:06:27	53 s	13/13	125.0 MiB			388.9 MiB
61	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 22:57:18	41 s	11/11	259.4 MiB			1147.7 MiB
97	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:07:20	41 s	16/16	284.8 MiB		145.9 MiB	35.8 MiB
105	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:09:34	34 s	13/13	107.4 MiB			168.8 MiB
98	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:07:20	33 s	16/16	455.7 MiB			77.5 KiB
78	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 23:05:45	28 s	16/16			70.4 MiB	1184.0 B
106	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:10:08	16 s	16/16			168.8 MiB	8.9 MiB
15	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 22:56:39	16 s	58/58	211.9 MiB	9.2 MiB		
65	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:02:52	15 s	16/16	1178.9 MiB	27.3 MiB		
99	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:09:16	11 s	160/160	1111.6 KiB		774.8 KiB	2.4 MiB
13	Listing leaf files and directories for 1826 paths: file:/home/naim/episcope/data/landing/stock/stock_20210101.jsonl, ... json at NativeMethodAccessorImpl.java:0 +details	2026/10/31 22:56:26	10 s	1826/1826				
0	Listing leaf files and directories for 1826 paths: file:/home/naim/episcope/data/landing/encounters/encounters_20210101.csv, ... csv at NativeMethodAccessorImpl.java:0 +details	2026/10/31 22:53:44	9 s	1826/1826				
69	collect at /home/naim/episcope/pipeline/silver/transform.py:147 +details	2026/10/31 23:05:31	5 s	16/16	1178.9 MiB			5.7 KiB
102	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:09:17	4 s	16/16			35.8 MiB	1365.2 KiB
118	parquet at <unknown>:0 +details	2026/10/31 23:10:33	4 s	16/16			41.6 MiB	52.2 KiB

Figure 5. Spark UI Stages, optimisations off (SF 0.5); the slowest stage is silver de-duplication (4.8 min, 1.15 GB shuffle).



▾ Completed Stages (128)

Page: 1 2 > 2 Pages. Jump to 1 . Show 100 items in a page. Go

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
76	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 23:33:57	4.3 min	16/16			1147.8 MiB	
85	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:38:30	3.1 min	16/16	966.8 MiB	1056.7 MiB		
2	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:30:27	2.2 min	67/67	1278.7 MiB	256.9 MiB		
125	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:42:54	1.1 min	13/13	125.0 MiB			388.9 MiB
127	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:44:02	1.1 min	16/16			388.9 MiB	
149	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:45:28	33 s	13/13	107.4 MiB			168.8 MiB
101	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 23:42:04	30 s	16/16			70.4 MiB	
74	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 23:33:29	27 s	11/11	259.4 MiB			1147.8 MiB
128	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:44:02	24 s	16/16	455.7 MiB			
151	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:46:01	17 s	16/16			168.8 MiB	8.9 MiB
132	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:45:07	12 s	16/16	455.7 MiB			9.9 MiB
15	Listing leaf files and directories for 1826 paths: file:/home/naim/episcope/data/landing/stock/stock_20210101.jsonl, ... json at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:32:55	10 s	1826/1826				
83	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:38:19	10 s	16/16	966.8 MiB	27.3 MiB		
17	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:33:07	8 s	58/58	211.9 MiB	9.2 MiB		
0	Listing leaf files and directories for 1826 paths: file:/home/naim/episcope/data/landing/encounters/encounters_20210101.csv, ... csv at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:30:15	8 s	1826/1826				
133	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:45:07	8 s	16/16	455.7 MiB			35.8 MiB
87	collect at /home/naim/episcope/pipeline/silver/transform.py:147 +details	2026/10/31 23:41:40	7 s	16/16	966.8 MiB			5.7 KiB
110	parquet at NativeMethodAccessorImpl.java:0 +details	2026/10/31 23:42:38	6 s	16/16	85.1 MiB	11.8 MiB		
99	first at /home/naim/episcope/pipeline/silver/transform.py:95 +details	2026/10/31 23:41:58	6 s	10/10	9.5 MiB			70.4 MiB

Figure 6. Spark UI Stages, optimisations on (SF 0.5); the same stage takes 4.3 min.

Task	P1	P2	What happened
1 Find a district with a dengue alert	Yes, 10 s	Yes, 10 s	Both named Jashore and Cox's Bazar from the red triangles at once
2 Did Dhaka dengue rise or fall in the last 6 months?	Partly, 20 s	No	P1 described the July–October rise and fall but not the recent direction; P2: "I don't understand graphs well"
3 What does the shaded band on the forecast mean?	Yes, 30 s	No	P1: "the likely range of future patients" (first called it a "data range"); P2 could not say
4 Why are some districts striped?	Yes, 40 s	Yes, 40 s	"Data not published" / "information kept hidden"
5 What changes when you move the budget slider?	Yes, 20 s	Yes, 40 s	Both: "the supplies change"

%) are the weakest cases.

- **Not built.** Delta Lake time travel, streaming, role-based access control and the SEIR scenario module (ING-5 to 7, DQ-4, PRV-3/4, AN-10, VIS-7).

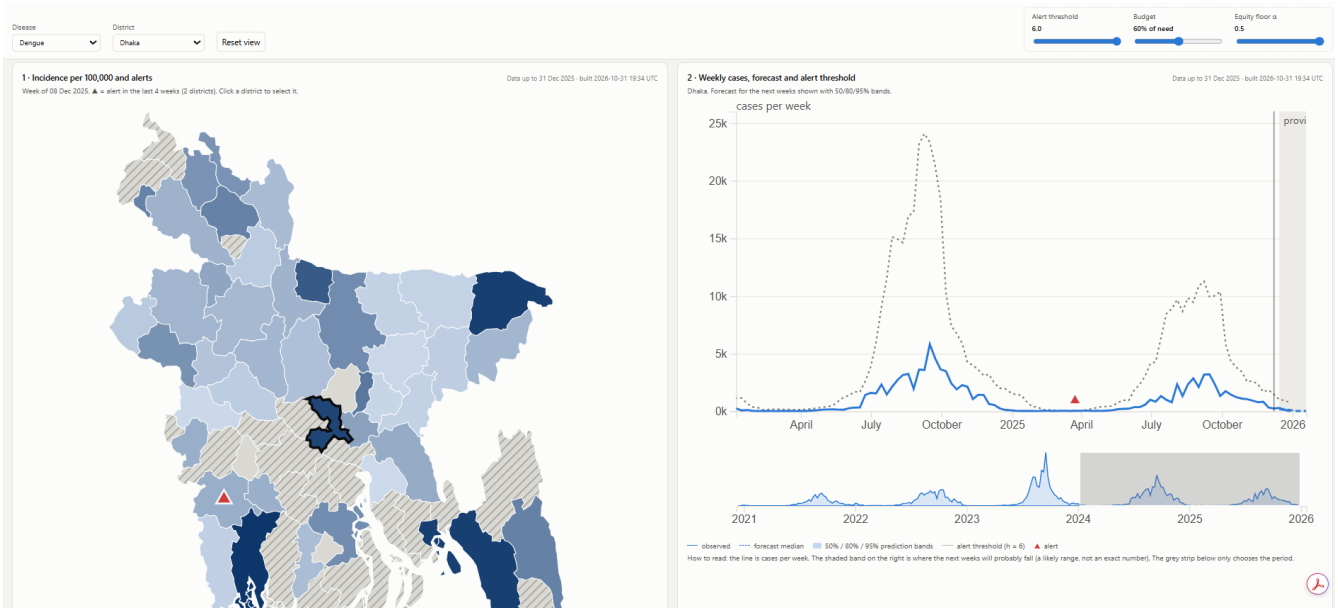


Figure 7. Dashboard with Dhaka selected: linked choropleth map with alerts and suppressed districts (left), weekly cases with forecast bands and alert threshold (right), what-if sliders (top).

Risk	Who is affected	Control in EpiScope
Re-identification of patients	Patients, especially in small districts	Salted hashing, age bands, identifiers kept in bronze, district-level output, suppression of 1–4 with complementary suppression [8]
Stigma for districts flagged as outbreak areas	Residents, local economy	Alerts framed as "unusual rise, investigate", not "outbreak"; access to alerts intended for health officials
False alarms (alert fatigue) and missed outbreaks	District officers, public	Threshold chosen from a measured trade-off (C1), adjustable, with sensitivity and false-alert rates published
Unfair allocation	Low-risk and remote districts	Equity floor α and a published efficiency–equity curve (C3); every assumption listed in one file
Over-trust in models	Policymakers	Forecasts always shown as ranges; methods are interpretable; synthetic-data banner
Lessons from "big data hubris" [4]	Everyone	Every method compared with a simple baseline before it is trusted

References

[1] L. Hutwagner, W. Thompson, G. M. Seeman, T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," *J. Urban Health*, 80(1), 2003.

[2] C. P. Farrington, N. J. Andrews, A. D. Beale, M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," *J. Royal Statistical Society A*, 159(3), 1996.

[3] J. Bracher, E. L. Ray, T. Gneiting, N. G. Reich, "Evaluating epidemic forecasts in an interval format," *PLoS Computational Biology*, 17(2), 2021.

[4] D. Lazer, R. Kennedy, G. King, A. Vespignani, "The parable of Google Flu: traps in big data analysis," *Science*, 343(6176), 2014.

[5] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Commun. ACM*, 59(11), 2016.

[6] M. Armbrust et al., "Delta Lake: High-performance ACID table storage over cloud object stores," *PVLDB*, 13(12), 2020.

[7] M. Bostock, V. Ogievetsky, J. Heer, "D3: Data-Driven Documents," *IEEE TVCG*, 17(12), 2011.

[8] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5), 2002.

[9] World Health Organization, *Technical Guidelines for Integrated Disease Surveillance and Response (IDSR)*, 3rd ed.

[10] geoBoundaries (CC BY 4.0); Open-Meteo historical weather API (CC BY 4.0); PuLP and CBC; statsmodels; scikit-learn.

[11] IFRC, "Bangladesh: Dengue Response Operation 2023, DREF Operation MDRBD031, Final Report," ReliefWeb, 2024. [Online]. Available: https://reliefweb.int/report/bangladesh/bangladesh-dengue-response-operation-2023-dref-operation-mdrbd031-final-report

Appendix A: Policy brief

To: Director, Disease Control and Preparedness Unit. **From:** EpiScope team. **Data up to:** 31 December 2025 (plan for the four weeks from 5 January 2026).

1. The situation now. The 2025 dengue season has ended nationally, but two districts show an unusual late rise that local teams should check: Cox's Bazar (50 cases in the week of 8 December against about 11 expected) and Jashore (40 against about 10 in the week of 17 November). Dengue should stay low in January, at roughly 375 cases nationwide over the coming weeks. Diarrhoeal disease is the next concern: we expect roughly 52,000 cases nationwide in the coming weeks (likely range about 35,000 to 75,000), rising towards the pre-monsoon season from mid-February. Stocks of dengue test kits (NS1) are sufficient everywhere; the shortfalls are in oral rehydration salts (ORS) and IV saline, largest in Dhaka, Gazipur, Chattogram, Cumilla and Noakhali.

2. Our three recommendations.

1. Send ORS to Dhaka, Gazipur, Chattogram, Cumilla and Noakhali, and IV saline to Chattogram, Noakhali, Cox's Bazar, Tangail and Gaibandha, by 19 January 2026. No new NS1 kits are needed now. Using our plan instead of sharing by population would cut unmet need from 24 % to 4 % for the same money.
2. Start safe-water and ORS awareness for diarrhoea in all divisions by 5 January 2026. Then start mosquito larval-control campaigns by 27 April 2026 in Dhaka, Khulna, Mymensingh, Rajshahi and Rangpur, and by 4 May in Barishal, Chattogram and Sylhet, about six weeks before the usual dengue rise, with NS1 kits and IV saline in place by mid-May.
3. Guarantee every district a minimum share of supplies (our "fairness floor" of 0.5). At this budget it costs almost nothing in total (0.1 %), and on a tight budget it is what stops most districts receiving nothing.

3. How to read the dashboard. A darker colour means more cases per 100,000 people. A triangle means a rise larger than usual that should be checked. A shaded band shows the likely range of future cases. Striped areas are hidden on purpose because the numbers are too small to publish safely.

4. How sure are we. Warnings caught about 87 % of test outbreaks, with about one false warning per district per year, usually within three weeks. Small local outbreaks and malaria are caught less often. Forecasts are ranges, not promises. The system was built and tested on realistic made-up data, so it should be checked on real reports before use.

5. Protecting people's data. No names or ID numbers leave the secure first layer. Patient IDs are scrambled with a secret key, ages are grouped, and any count below five is hidden from published maps and tables.

Appendix B: Contribution matrix

Agree the shares together, then everyone signs. L = lead, S = support, – = none.

Work area	Naim	Shamraj	Ratul	Puja
Problem analysis, proposal, conflict analysis	L	S	S	S
Synthetic generator and public-data acquisition	L	–	–	–
Ingestion, lakehouse layers, incremental loads	L	–	–	S (tests)
Data quality, lineage and privacy controls	L	–	S (suppression)	–
Outbreak detection and evaluation	–	L	–	S (tests)
Forecasting, backtesting and demand conversion	–	L	–	S (tests)
Optimisation model and preparedness calendar	–	L	–	–
Serving layer and D3.js dashboard	S	–	L	–
Testing and usability study	S	–	–	L
Scalability experiment and Spark optimisations	L	–	–	–
Report and policy brief	L	S	S	S
Video production	L	S	S	S
Agreed overall share (%, total 100)	25	25	25	25
Commits (git shortlog -sn)	13	4	3	6
Signature	Naim, 02-11-2026	Shamraj, 02-11-2026	Ratul, 02-11-2026	Puja, 02-11-2026

This is a four-member group with the instructor's approval; the instructor confirmed that the contribution factor is $\pi = \text{clip}(4\sigma/100, 0.8, 1.1)$, so equal shares of 25 % give $\pi = 1$.

Appendix C: Individual reflections

Each member writes their own half page: what they built, what was hardest, what they learned, what they would do differently.

MD Naim Uddin Mozumdar. In this project I was responsible for data engineering: the data generator, the data pipeline, the data-quality rules, and leading the overall project. The most challenging part was setting up WSL and Spark for the first time and bringing the SF 1 run under 45 minutes; I solved this by optimising the gold layer with column pruning. I learned about Spark performance optimisation, data-pipeline management and the importance of memory, especially with only 8 GB of RAM. Working in a team taught me to coordinate tasks, manage responsibilities, collaborate and integrate everyone's contributions. If I did this project again, I would start earlier, manage my time better, and finish the video and the final report well before the deadline.

Shamraj Deb Nath. I was responsible for analytics engineering: outbreak detection with EARS C2 and historical limits, forecasting with a Poisson GLM, demand conversion, budget allocation with a linear-programming optimiser, and the preparedness calendar; I also presented Chapters 4 and 7 of the video. The most challenging part was balancing detection against false alerts. EARS C2 generated too many false alerts, so I chose historical limits, which detected 87 % of outbreaks with about one false alert per district per year. I learned that a model is only useful when it beats a simple baseline, and that understandable models and clear policy recommendations matter as much as accuracy. Teamwork taught me the value of communication, shared responsibility and combining different ideas. Next time I would improve the detection of small local outbreaks and malaria, plan allocation over several weeks instead of one, and keep more regular contact with the team.

Syed Nasibur Rahaman Ratul. I was responsible for visualisation engineering: the D3.js dashboard with five linked views, the what-if sliders, the JSON serving layer and privacy suppression; I also presented Chapters 1 and 5 of the video. The most challenging part was complementary suppression: hiding only counts from 1 to 4 was not enough, because users could recover them from published totals, so additional cells had to be suppressed. I learned that a dashboard must be easy to understand, not only attractive, and gained experience in data visualisation, privacy protection and showing forecast uncertainty as a range instead of a single number; the usability results confirmed this. Teamwork taught me to coordinate tasks, share ideas and integrate different components into one system. Next time I would involve non-technical users from the start, add simple "rising / falling" explanations beside the graphs, and make the dashboard work well on mobile.

Puja Dey. I was responsible for testing and quality assurance. I added four test files with 23 tests covering checksum idempotency, forecast calculations, EARS C2 detection and detection metrics, ran the dashboard usability test with two non-CS participants, and presented Chapter 6 of the video. The most challenging part was joining the team late, when most of the code already existed; I had to understand other members' code before I could write meaningful tests, which I did by reviewing it carefully and testing its logic. I learned that testing is not only about whether code runs but about proving that it is correct, and I learned the value of usability testing and of checking AI-drafted test cases myself; seeing all 52 tests pass gave me confidence in the system. Teamwork taught me to adapt to an existing workflow and take responsibility for quality. Next time I would join from the beginning, write tests alongside the code, test with more users, and run the tests automatically on every push with GitHub Actions.